

SPRING BOOT & JPA MASTER NOTES

Comprehensive Placement & Enterprise Development Guide

1. What is Spring Boot?

Spring Boot is an open-source Java-based framework built on top of Spring that simplifies configuration, setup, and deployment using auto-configuration, embedded Tomcat servers, and opinionated starter dependencies.

2. Key Module Topics Covered

- Module 1: Spring vs Spring Boot & Embedded Containers (JAR vs WAR)
- Module 2: IoC Containers & Dependency Injection (Constructor, Setter, Field)
- Module 3: Key Annotations (@RestController, @Service, @Repository, @Configuration)
- Module 4: Spring Data JPA, ORM Mappings & Entity Lifecycles
- Module 5: SOLID Design Principles applied in Spring Architecture
- Module 6: Spring Bean Lifecycle (@PostConstruct, BeanPostProcessor, @PreDestroy)
- Module 7: Aspect-Oriented Programming (AOP), Advice Types & Proxies (JDK vs CGLIB)
- Module 8: Spring Boot Actuator, Component Scanning & Profile Management

3. Sample Code: RestController & Constructor Injection

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService; // Recommended Constructor Injection
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }
}
```

4. Spring Data JPA & Entity Lifecycle States

1. TRANSIENT: Object exists only in JVM memory; Hibernate is unaware of it.
2. PERSISTENT: Object is managed by EntityManager / Session and tracked for changes.
3. DETACHED: Session closed; object exists in JVM but changes won't sync to DB.
4. REMOVED: Entity is marked for deletion from the database.

5. Aspect Oriented Programming (AOP) Cheat Sheet

- Aspect: A class containing cross-cutting concerns (Logging, Security, Transactions).
- Advice: Defines WHAT action to perform (@Before, @After, @AfterReturning, @Around).
- Pointcut: Defines WHERE the advice should be applied using expressions.
- Join Point: Execution point in application where advice can be plugged in.
- Proxies: JDK Dynamic Proxy (for Interfaces) vs CGLIB Proxy (for Subclasses).